

SwipeLab Testing Strategy & Documentation

1. Overview

The SwipeLab project employs a comprehensive full-stack testing strategy designed to ensure correctness, reliability, and maintainability across both backend and frontend systems.

- **Backend:** A heavy emphasis on Unit and Integration testing to ensure data integrity, complex gamification logic, and cross-domain eventing.
- **Frontend:** End-to-End (E2E) testing to ensure user flows (like swiping, viewing stats, and admin management) work flawlessly from a user's perspective.

Because the application consists of a Java/Spring Boot backend and a React Native frontend, testing is divided into two complementary layers:

Backend Testing

The backend focuses primarily on Unit Tests and Integration Tests to validate:

- Domain business logic
- Authentication and authorization
- Task management workflows
- Classification processing
- Gamification calculations
- Event-driven communication between domains
- External integrations such as StarDBI

These tests ensure that individual components behave correctly and that cross-domain workflows function as expected (400 tests).

Frontend Testing

The frontend follows a User Story Driven End-to-End (E2E) testing strategy implemented with Playwright.

Rather than testing individual screens in isolation, the E2E suite validates complete user journeys from the perspective of real application users. Each major functional requirement is represented by an independent automated scenario covering UI interactions, API communication, navigation, permissions, and expected business outcomes.

The E2E suite is organized around three application roles:

- Regular Users (U1–U9)
- Researchers (R1–R8)
- Administrators (A1–A3)

Together these scenarios provide full coverage of the platform's primary workflows, including authentication, task participation, image classification, analytics, gamification, researcher operations, administrative management, and external system integrations.

To improve maintainability and contributor onboarding, every E2E scenario is tracked as an independent GitHub issue under the E2E Test Suite Epic (#293). This allows contributors to implement, review, and maintain tests independently while ensuring complete coverage of all supported user stories.

The combination of backend unit/integration testing and frontend end-to-end testing provides confidence that both individual system components and complete user workflows operate correctly in production-like environments.

2. Backend Testing (Java / Spring Boot)

2.1 Tools & Frameworks

While the exact dependencies are managed in pom.xml, the standard SwipeLab backend testing stack includes:

- **JUnit 5:** The core testing framework for test execution and lifecycle management.
- **Mockito:** Used for mocking dependencies (Repositories, external services) to isolate unit tests.
- **Spring Boot Test:** Provides context loading and utilities like MockMvc for API endpoint testing.
- **H2 / Testcontainers (Inferred):** Managed via application-test.properties and application-integration.properties for testing against temporary databases without affecting production.

2.2 Package Structure

The backend test suite strictly mirrors the **Domain-Driven Design (DDD)** of the src/main/java folder. Tests are grouped by domain (auth, classification, gamification, recipients, tasks, users), and further subdivided by architectural layers:

- api/ (Controllers)
- application/ (Services, Event Listeners)

- domain/ (Core logic, Calculators, Mappers)
- infrastructure/ (Security, Configuration)

2.3 Types of Backend Tests

A. Domain & Utility Unit Tests (Isolated)

These tests verify core business logic without loading the Spring Context. They are lightning-fast and mock all external dependencies.

- **Calculators/Logic:** Tests like `CredibilityCalculatorTest.java` verify the math behind user scoring.
- **Mappers:** Tests like `AuthMapperTest.java` and `TaskMapperTest.java` ensure Data Transfer Objects (DTOs) map correctly to Entities.

B. Application Service Tests

These test the business use cases. Dependencies like repositories are mocked to ensure the service logic (e.g., fetching images, calculating streaks) behaves correctly.

- *Examples:* `ClassificationServiceTest.java`,
`GamificationOrchestratorServiceTest.java`,
`TaskDistributionServiceTest.java`.

C. API / Controller Tests

These tests use `MockMvc` to simulate HTTP requests and verify routing, status codes, JSON serialization, and input validation without needing a running server.

- *Examples:* `AuthControllerTest.java`, `ClassificationControllerTest.java`,
`GamificationControllerTest.java`.

D. Integration Tests

These are crucial for `SwipeLab`, as the app relies on event-driven architecture (e.g., classifying an image triggers gamification updates). These tests load a larger Spring context to verify that different domains talk to each other correctly.

- *Examples:* * `ClassificationToGamificationIntegrationTest.java`: Ensures a swipe results in points/badges.
 - `FraudDetectionFlowIntegrationTest.java`: Ensures bad classifications properly flag users.
 - `GamificationSyncIntegrationTest.java`: Verifies end-to-end sync logic.
-

3. Frontend Testing (React Native / Expo)

3.1 Tools & Frameworks

- **Playwright:** The primary framework used for Frontend testing, configured via `frontend/playwright.config.ts`.
- *(Note: The current structure heavily favors E2E testing over isolated component testing like Jest, ensuring the app works holistically).*

3.2 Types of Frontend Tests

The frontend test suite (located in `frontend/tests/e2e/`) focuses on complete End-to-End (E2E) user journeys executed through Playwright. These tests launch the application and interact with it exactly as a real user would, validating UI behavior, API integration, navigation, permissions, and business workflows.

The E2E suite is organized by application role and user story.

A. Regular User (Swiper) Flows

These tests validate the primary user experience of browsing tasks, classifying images, earning rewards, and tracking progress.

ID	Test Scenario
U1	Register & Login
U2	Browse Available Tasks
U3	Join a Task
U4	Swipe / Classify Images
U5	View Challenges
U6	View Statistics
U7	View Collection
U8	Gamification & Leaderboards
U9	Species Reference Images

Example test files:

`tests/e2e/user/register-login.spec.ts`

tests/e2e/user/browse-tasks.spec.ts

tests/e2e/user/join-task.spec.ts

tests/e2e/user/swipe-images.spec.ts

tests/e2e/user/challenges.spec.ts

tests/e2e/user/statistics.spec.ts

tests/e2e/user/collection.spec.ts

tests/e2e/user/gamification.spec.ts

tests/e2e/user/species-reference-images.spec.ts

B. Researcher Flows

These tests validate the workflows used by researchers to create, manage, monitor, and export classification tasks.

ID	Test Scenario
R1	StarDBI Login
R2	Create Task from Experiment
R3	View Dashboard
R4	Manage Gold Images
R5	View Analytics
R6	Manage Recipients
R7	Export Data
R8	Pause / Archive Task

Example test files:

tests/e2e/researcher/stardbi-login.spec.ts

tests/e2e/researcher/create-task.spec.ts

tests/e2e/researcher/dashboard.spec.ts

tests/e2e/researcher/gold-images.spec.ts

tests/e2e/researcher/analytics.spec.ts

tests/e2e/researcher/recipients.spec.ts

tests/e2e/researcher/export-data.spec.ts

tests/e2e/researcher/pause-archive-task.spec.ts

C. Administrator Flows

These tests validate platform administration and moderation capabilities.

ID	Test Scenario
A1	User Management
A2	Suspicious Activity Review
A3	Notifications

Example test files:

tests/e2e/admin/user-management.spec.ts

tests/e2e/admin/suspicious-activity.spec.ts

tests/e2e/admin/notifications.spec.ts

D. E2E Test Architecture

All E2E tests are executed using a dedicated testing environment that includes:

Seeded test users for each role (USER, RESEARCHER, ADMIN)

Mock StarDBI integration

Deterministic test datasets

Automated Playwright fixtures

Isolated test execution to prevent cross-test contamination

Each user story corresponds to an independent GitHub issue linked to the E2E Test Suite Epic (#293), allowing contributors to implement and maintain tests in parallel.

4. Testing Conventions & Guidelines

To maintain code quality, the team follows these conventions:

1. Naming Conventions:

- **Backend:** All test classes must end with Test.java (e.g., UserServiceTest.java).
- **Frontend:** All Playwright tests must end with .spec.ts (e.g., admin-tasks.spec.ts).

2. Environment Separation:

- The backend utilizes application-test.properties for fast, in-memory unit tests, and application-integration.properties for heavier context loads.

3. Mocking External Systems:

- Any third-party integration (like the StardbClient and EmailService) must be mocked during standard unit tests to prevent network failures from causing flaky tests.

4. Test Data:

- Rely on factories or the MockDataSeeder logic when generating test data to avoid brittle tests dependent on hardcoded database states.

5. CI/CD Guardrails:

- *(Best Practice based on Git workflow):* All PRs targeting the main branch must pass the full suite of backend Unit Tests and frontend Playwright specs before they can be merged.